

Lógica Digital - Circuitos Secuenciales

Segundo Cuatrimestre 2025

Sistemas Digitales

DC - UBA

Introducción

Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va ser la siguiente:

- **Latches**
- Flip-flops
- Metaestabilidad
- Registros y memorias

Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va ser la siguiente:

- **Latches**
- **Flip-flops**
- Metaestabilidad
- Registros y memorias

Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va ser la siguiente:

- **Latches**
- **Flip-flops**
- **Metaestabilidad**
- Registros y memorias

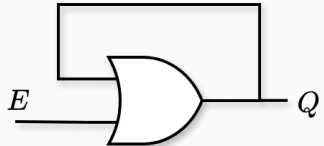
Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va ser la siguiente:

- **Latches**
- **Flip-flops**
- **Metaestabilidad**
- **Registros y memorias**

Latches

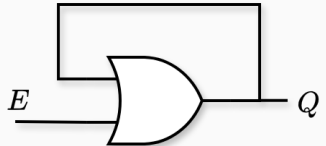
Retroalimentación

- Asumamos que tanto la entrada E como la salida Q valen 0 en este instante.



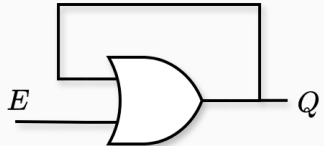
Retroalimentación

- Asumamos que tanto la entrada E como la salida Q valen 0 en este instante.
- ¿Qué pasa si ahora E cambia a 1?

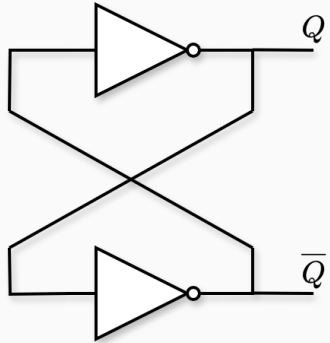


Retroalimentación

- Asumamos que tanto la entrada E como la salida Q valen 0 en este instante.
- ¿Qué pasa si ahora E cambia a 1?
- Y luego... ¿si E vuelve a 0?

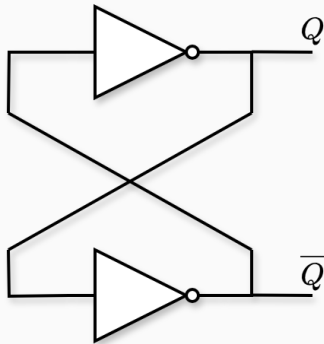


Bistable



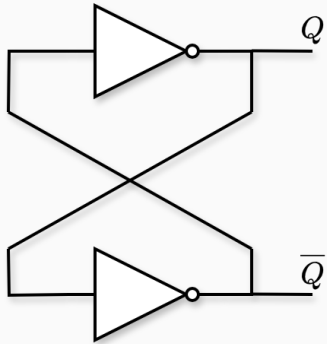
Biestable

- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$



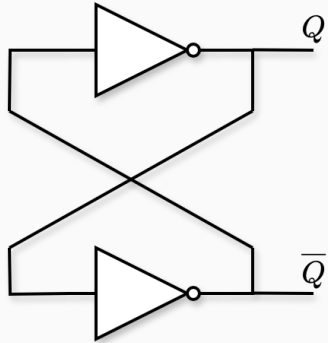
Biestable

- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$



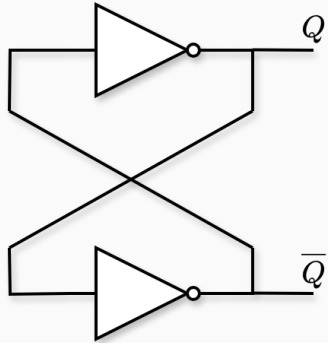
Biestable

- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de biestable



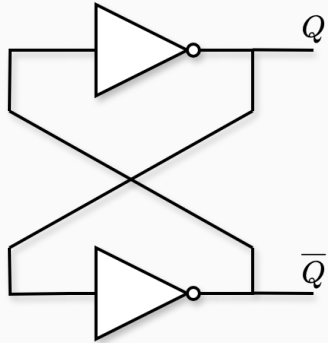
Biestable

- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de biestable
- Decimos que este circuito tiene **memoria**.



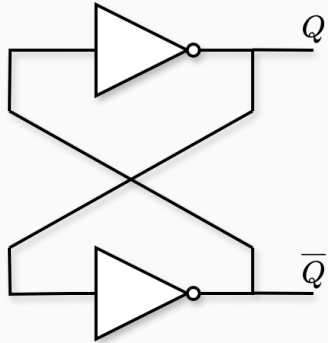
Biestable

- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de biestable
- Decimos que este circuito tiene **memoria**.
- **No** podemos controlar el estado de la memoria.



Biestable

- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de biestable
- Decimos que este circuito tiene **memoria**.
- **No** podemos controlar el estado de la memoria.



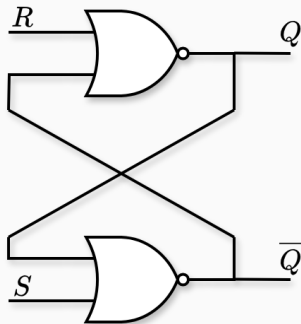
Hay un tercer estado *metaestable*, que veremos más adelante.

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .



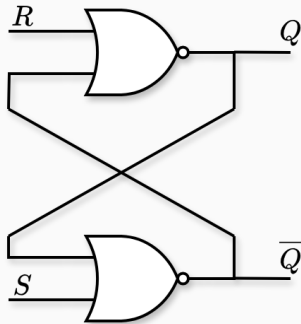
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0		
0	0		
0	1		
1	1		



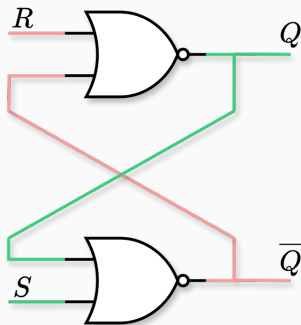
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0	1	0
0	0		
0	1		
1	1		



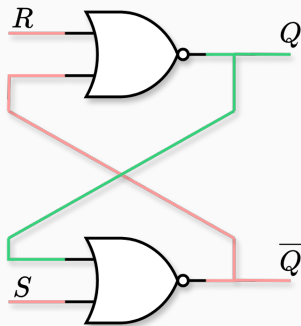
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0	1	0
0	0	Q_{prev}	$\overline{Q}_{prev}$
0	1		
1	1		



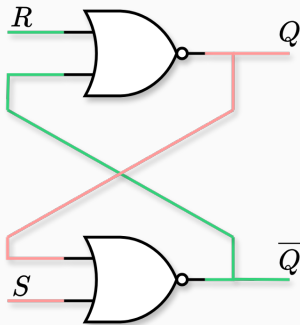
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0	1	0
0	0	Q_{prev}	$\overline{Q}_{prev}$
0	1	0	1
1	1		



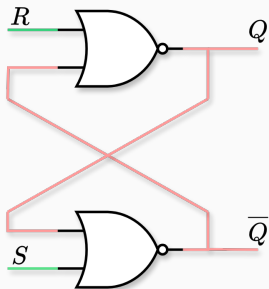
A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

Latch SR

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

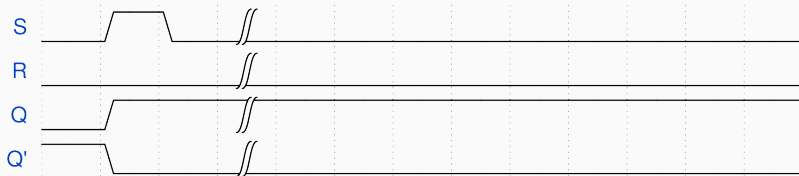
S	R	Q	$\overline{Q}$
1	0	1	0
0	0	Q_{prev}	$\overline{Q}_{prev}$
0	1	0	1
1	1	0	0



A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

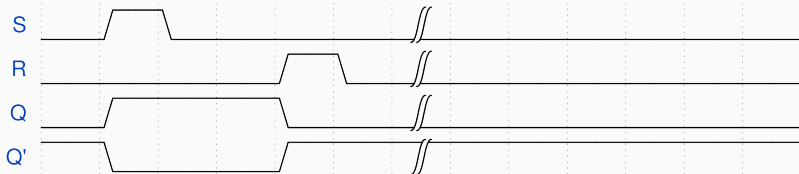
Latch SR

S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U



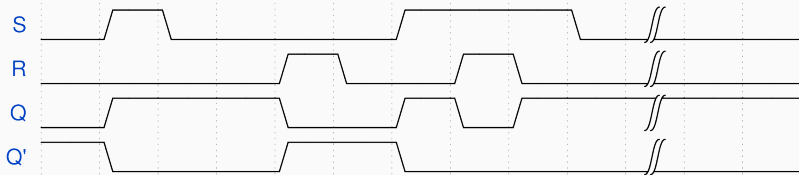
Latch SR

S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U



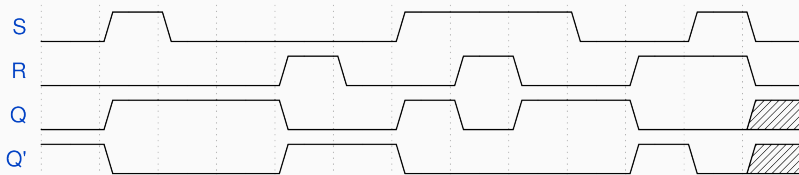
Latch SR

S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U

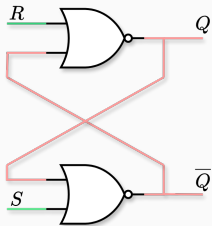


Latch SR

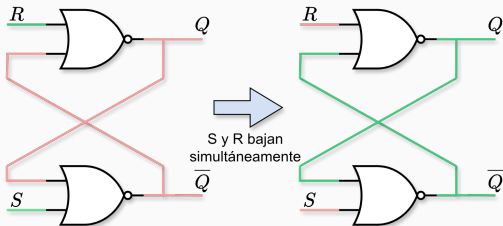
S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U



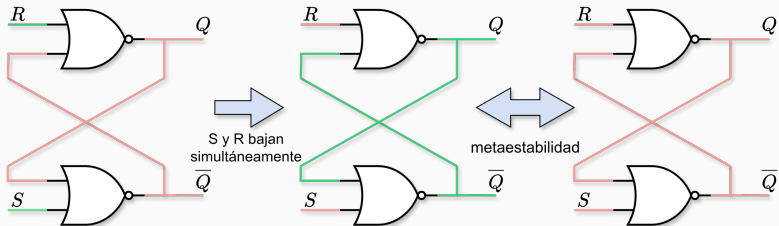
Latch SR. Metaestabilidad



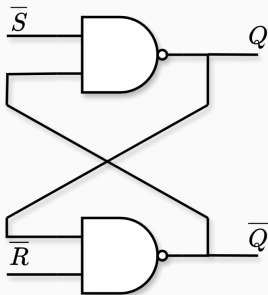
Latch SR. Metaestabilidad



Latch SR. Metaestabilidad

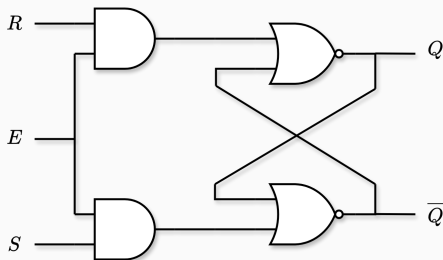


Latch SR. Contrucción con NANDs



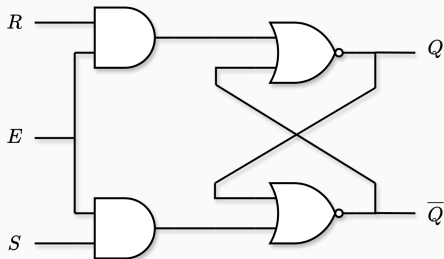
$\overline{S}$	$\overline{R}$	Q	$\overline{Q}$
1	1	latch	latch
1	0	0	1
0	1	1	0
0	0	U	U

Latch SR con Enable



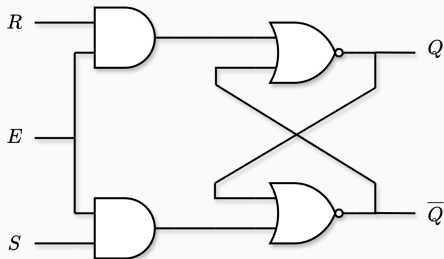
- El comportamiento es el mismo que el latch SR, pero solo se activa cuando E es 1.

Latch SR con Enable



- El comportamiento es el mismo que el latch SR, pero solo se activa cuando E es 1.
- Si la señal E es 0, entonces el latch se mantiene en su estado actual.

Latch SR con Enable



- El comportamiento es el mismo que el latch SR, pero solo se activa cuando E es 1.
- Si la señal E es 0, entonces el latch se mantiene en su estado actual.
- Si la señal E es un clock, entonces el latch se dice que es *level-sensitive*.

Latch JK

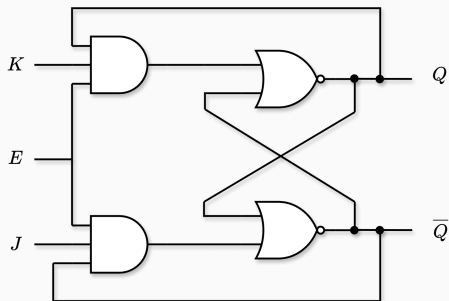


Tabla de verdad:

E	J	K	Q	$\bar{Q}$
1	1	0		
1	0	1		
1	0	0		
1	1	1		
0	-	-		

Latch JK

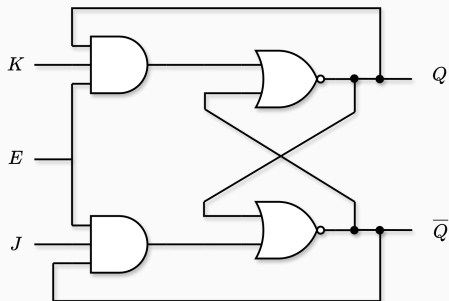


Tabla de verdad:

E	J	K	Q	$\bar{Q}$
1	1	0	1	0
1	0	1		
1	0	0		
1	1	1		
0	-	-		

Latch JK

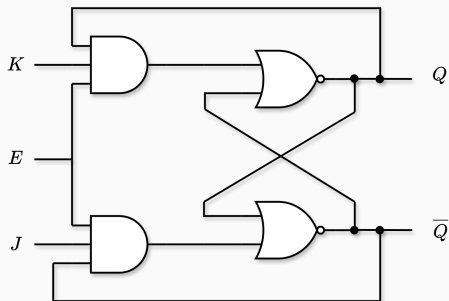


Tabla de verdad:

E	J	K	Q	$\bar{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0		
1	1	1		
0	-	-		

Latch JK

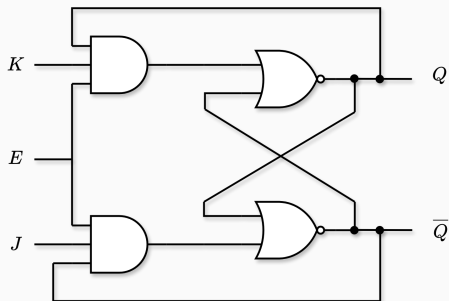


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	1	1		
0	-	-		

Latch JK

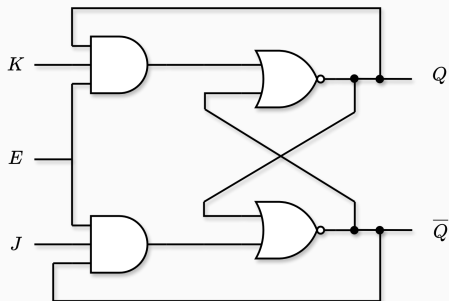


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	1	1	$\overline{Q}_{prev}$	Q_{prev}
0	-	-		

Latch JK

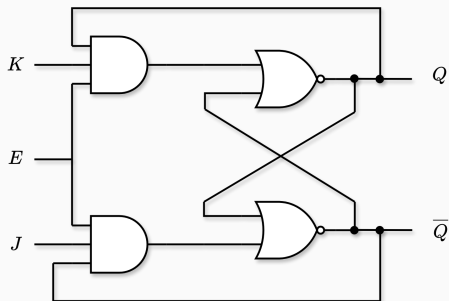


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	1	1	$\overline{Q}_{prev}$	Q_{prev}
0	-	-	Q_{prev}	$\overline{Q}_{prev}$

Latch JK

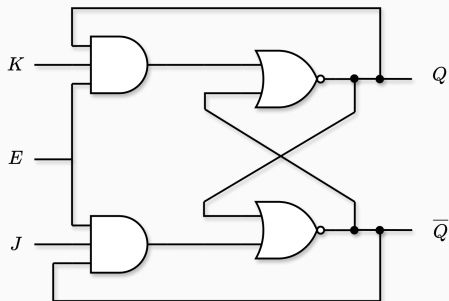


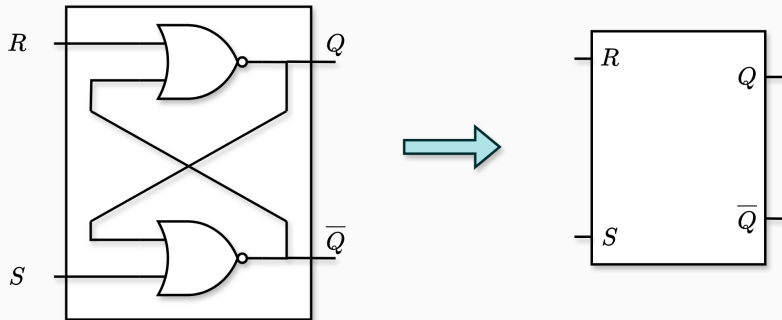
Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	1	1	$\overline{Q}_{prev}$	Q_{prev}
0	-	-	Q_{prev}	$\overline{Q}_{prev}$

Con $J, K = (1, 1)$:

- El valor de las salidas está ahora definido.
- El circuito oscila (estado inestable).

Notación



Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

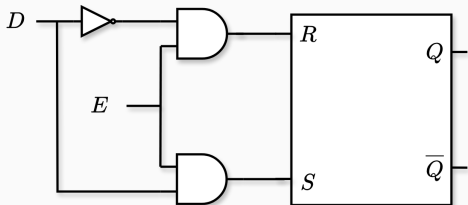


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-		
1	0		
1	1		

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

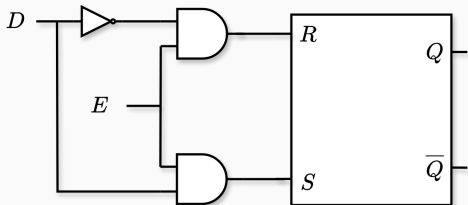


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-	Q_{prev}	$\overline{Q}_{prev}$
1	0		
1	1		

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

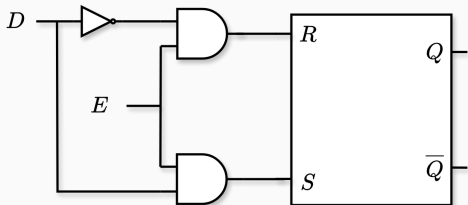


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-	Q_{prev}	$\overline{Q}_{prev}$
1	0	0	1
1	1		

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

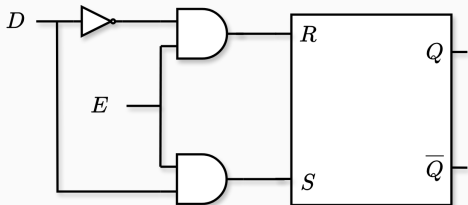


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-	Q_{prev}	$\overline{Q}_{prev}$
1	0	0	1
1	1	1	0

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

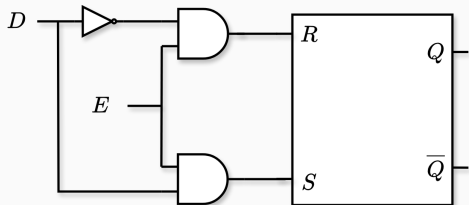
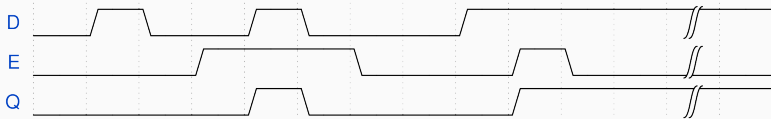


Tabla de verdad:

E	D	Q	$\bar{Q}$
0	-	Q_{prev}	$\bar{Q}_{prev}$
1	0	0	1
1	1	1	0



Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

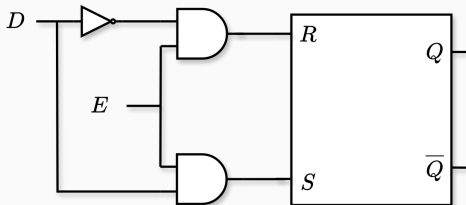


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-	Q_{prev}	$\overline{Q}_{prev}$
1	0	0	1
1	1	1	0

- D no debe cambiar durante una ventana de tiempo alrededor del flanco de bajada de E

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

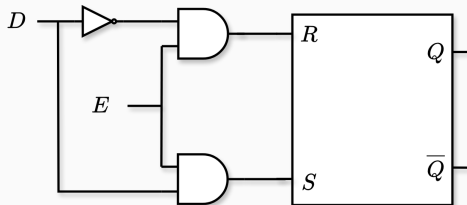


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-	Q_{prev}	$\overline{Q}_{prev}$
1	0	0	1
1	1	1	0

- D no debe cambiar durante una ventana de tiempo alrededor del flanco de bajada de E
- Los tiempos no se pueden predecir (dependen de D)

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

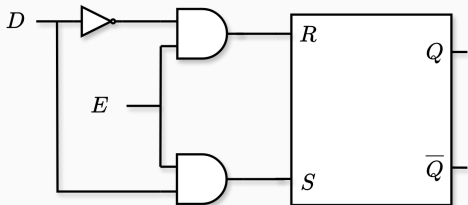


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	-	Q_{prev}	$\overline{Q}_{prev}$
1	0	0	1
1	1	1	0

- D no debe cambiar durante una ventana de tiempo alrededor del flanco de bajada de E
- Los tiempos no se pueden predecir (dependen de D)
- Puede causar carreras si existe un lazo en el circuito externo.

Flip-Flops

Sincronismo

En un sistema de cómputo necesitamos *sincronizar* los cambios de estado para que sucedan en el mismo momento y a intervalos regulares

Para eso necesitamos:

- Una señal de sincronismo (**clock**)

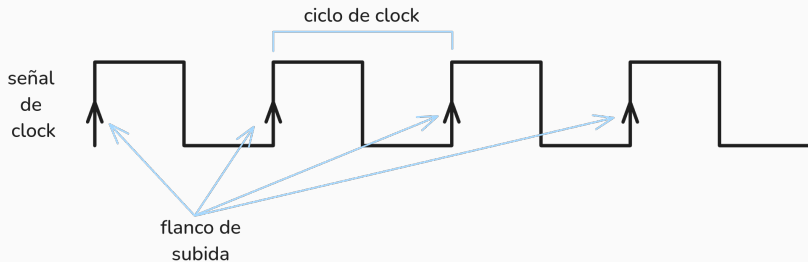
Sincronismo

En un sistema de cómputo necesitamos *sincronizar* los cambios de estado para que sucedan en el mismo momento y a intervalos regulares

Para eso necesitamos:

- Una señal de sincronismo (**clock**)
- Un dispositivo de memoria que no sea level-sensitive (**flip-flop**)

Señal de clock



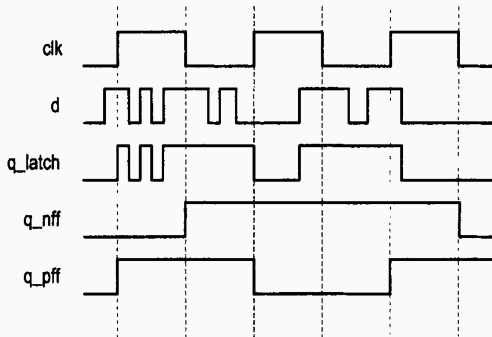
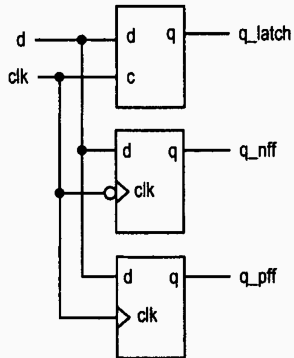
Período T : tiempo que tarda en completar un ciclo.

Frecuencia f : cantidad de ciclos por unidad de tiempo.

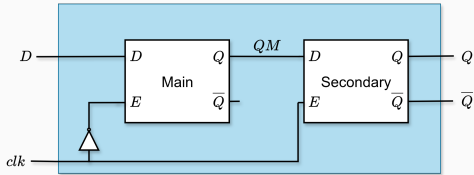
Son recíprocos:

$$f = \frac{1}{T}$$

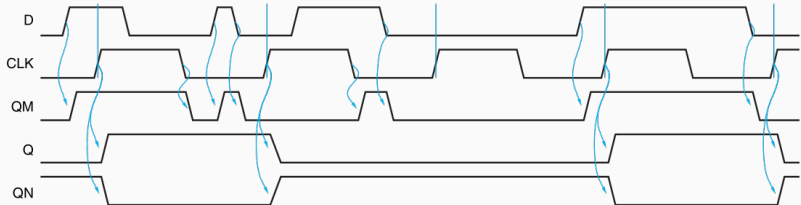
Sincronizando...



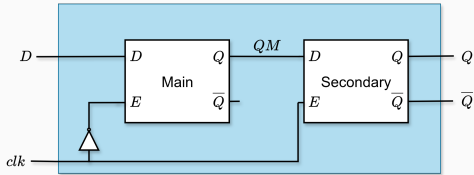
Flip-Flop D



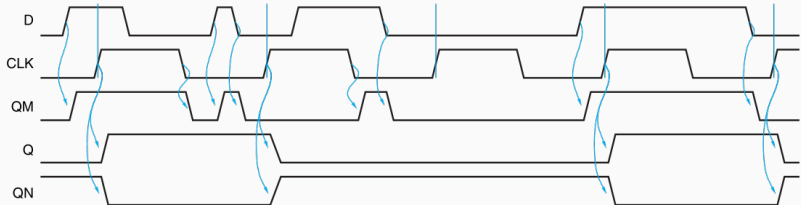
D	clock	Q	$\bar{Q}$
0			
1			
-	0		
-	1		



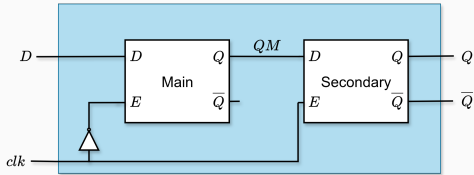
Flip-Flop D



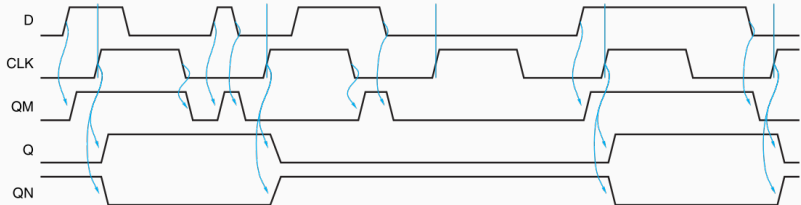
D	clock	Q	$\bar{Q}$
0		0	1
1			
-	0		
-	1		



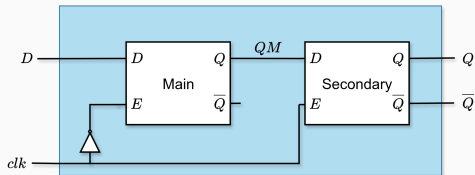
Flip-Flop D



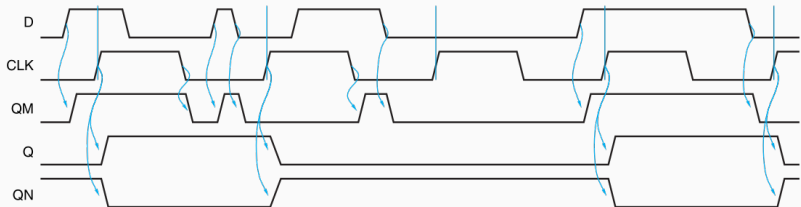
D	clock	Q	$\bar{Q}$
0		0	1
1		1	0
-	0		
-	1		



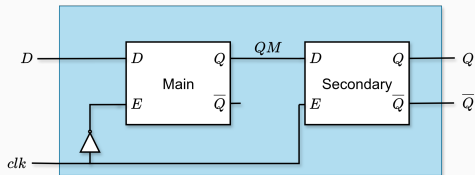
Flip-Flop D



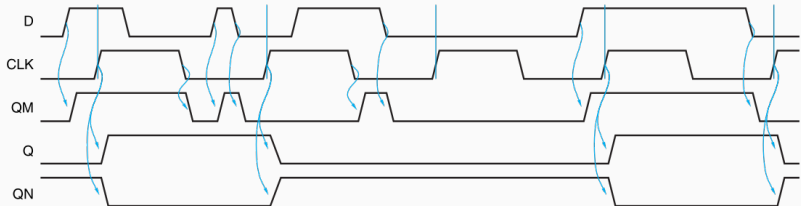
D	clock	Q	$\bar{Q}$
0		0	1
1		1	0
-	0	Q_{prev}	$\bar{Q}_{prev}$
-	1		



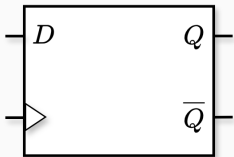
Flip-Flop D



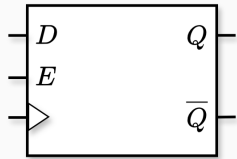
D	clock	Q	$\bar{Q}$
0		0	1
1		1	0
-	0	Q_{prev}	$\bar{Q}_{prev}$
-	1	Q_{prev}	$\bar{Q}_{prev}$



Flip-Flop D con Enable

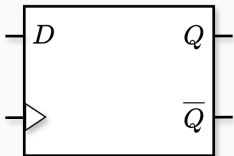


Flip-Flop D

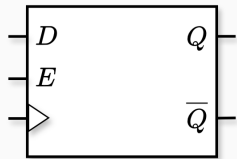


Flip-Flop D con Enable

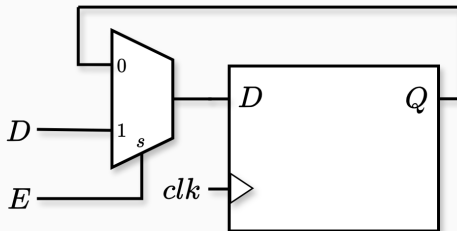
Flip-Flop D con Enable



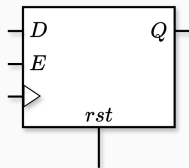
Flip-Flop D



Flip-Flop D con Enable

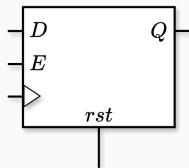


Reset



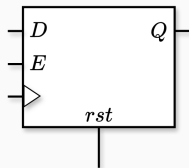
D	E	rst_a	rst_s	clk	Q
0	1	0	0		
1	1	0	0		
-	0	0	0		
-	-	0	1		
-	-	1	-	-	

Reset



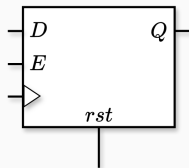
D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		
-	0	0	0		
-	-	0	1		
-	-	1	-	-	

Reset



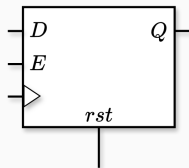
D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
-	0	0	0		
-	-	0	1		
-	-	1	-	-	

Reset



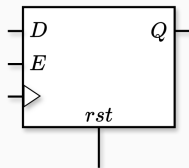
D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
-	0	0	0		Q_{prev}
-	-	0	1		
-	-	1	-	-	

Reset



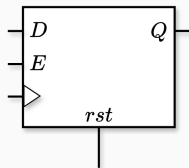
D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
-	0	0	0		Q_{prev}
-	-	0	1		0
-	-	1	-	-	

Reset

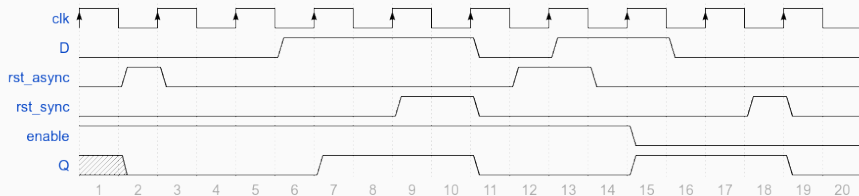


D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
-	0	0	0		Q_{prev}
-	-	0	1		0
-	-	1	-	-	0

Reset

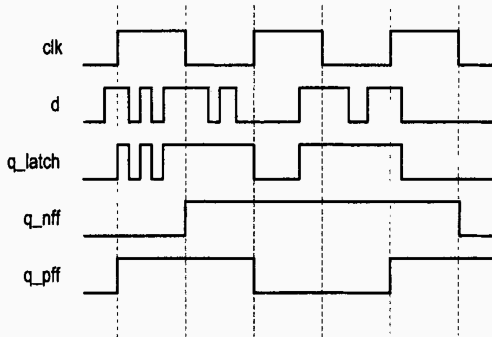
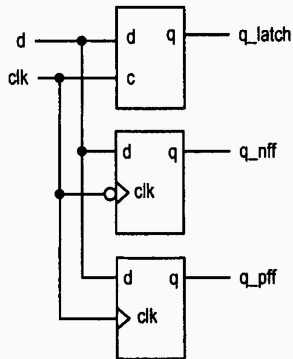


D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
-	0	0	0		Q_{prev}
-	-	0	1		0
-	-	1	-	-	0



Flip-Flop D

Ahora podemos entender bien las diferencias:



Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

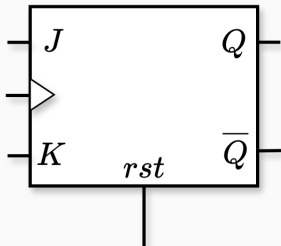


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$		
0	1	$1\uparrow$		
0	0	$1\uparrow$		
1	1	$1\uparrow$		
-	-	0		

Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

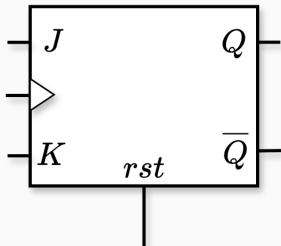


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$	1	0
0	1	$1\uparrow$		
0	0	$1\uparrow$		
1	1	$1\uparrow$		
-	-	0		

Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

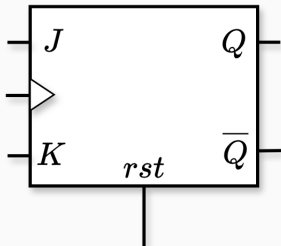


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$	1	0
0	1	$1\uparrow$	0	1
0	0	$1\uparrow$		
1	1	$1\uparrow$		
-	-	0		

Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

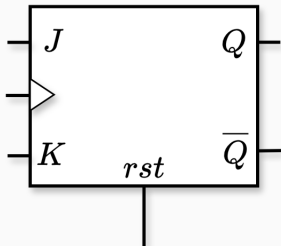


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$	1	0
0	1	$1\uparrow$	0	1
0	0	$1\uparrow$	Q_{prev}	$\overline{Q}_{prev}$
1	1	$1\uparrow$		
-	-	0		

Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

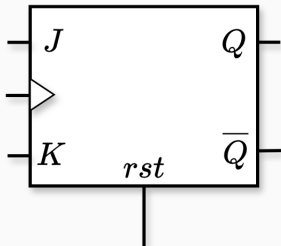


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$	1	0
0	1	$1\uparrow$	0	1
0	0	$1\uparrow$	Q_{prev}	$\overline{Q}_{prev}$
1	1	$1\uparrow$	$\overline{Q}_{prev}$	Q_{prev}
-	-	0		

Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

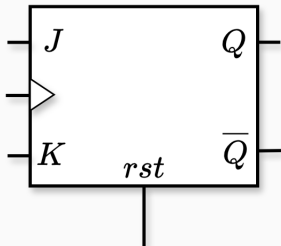


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$	1	0
0	1	$1\uparrow$	0	1
0	0	$1\uparrow$	Q_{prev}	$\overline{Q}_{prev}$
1	1	$1\uparrow$	$\overline{Q}_{prev}$	Q_{prev}
-	-	0	Q_{prev}	Q_{prev}

Flip-Flop JK

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

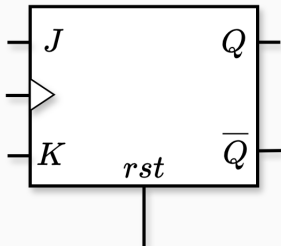


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	$1\uparrow$	1	0
0	1	$1\uparrow$	0	1
0	0	$1\uparrow$	Q_{prev}	$\overline{Q}_{prev}$
1	1	$1\uparrow$	$\overline{Q}_{prev}$	Q_{prev}
-	-	0	Q_{prev}	Q_{prev}

Ahora en el caso crítico donde $J, K = (1, 1)$ la salida tiene un estado y un tiempo de cambio bien definido:

Se niega el valor anterior cada 1 clock

Metaestabilidad

- En un flip-flop, la salida Q no cambia instantáneamente.

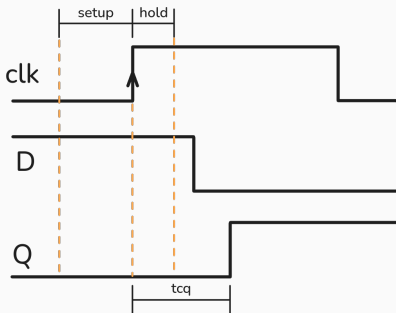
- En un flip-flop, la salida Q no cambia instantáneamente.
- No sólo eso, sino que la entrada debe permanecer estable durante una ventana de tiempo

Metaestabilidad

- En un flip-flop, la salida Q no cambia instantáneamente.
- No sólo eso, sino que la entrada debe permanecer estable durante una ventana de tiempo
- Si la entrada cambia durante esa ventana, la salida puede quedar en un estado indefinido.

Metaestabilidad

- En un flip-flop, la salida Q no cambia instantáneamente.
- No sólo eso, sino que la entrada debe permanecer estable durante una ventana de tiempo
- Si la entrada cambia durante esa ventana, la salida puede quedar en un estado indefinido.



Transición adecuada

- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana

Metaestabilidad

- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana
- La salida puede quedar en un estado **metaestable**

Metaestabilidad

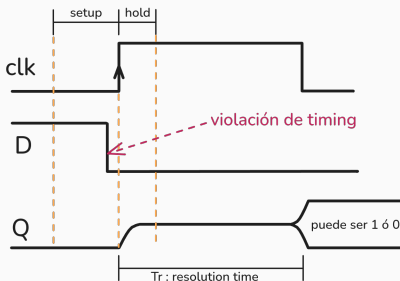
- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana
- La salida puede quedar en un estado **metaestable**
- Eventualmente, la salida se estabilizará en un estado válido, luego de un *tiempo de resolución* T_r .

Metaestabilidad

- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana
- La salida puede quedar en un estado **metaestable**
- Eventualmente, la salida se estabilizará en un estado válido, luego de un *tiempo de resolución* T_r .
- T_r es una variable aleatoria, y no podemos predecir su valor.

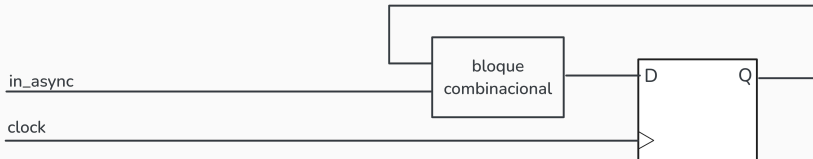
Metaestabilidad

- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana
- La salida puede quedar en un estado **metaestable**
- Eventualmente, la salida se estabilizará en un estado válido, luego de un *tiempo de resolución* T_r .
- T_r es una variable aleatoria, y no podemos predecir su valor.



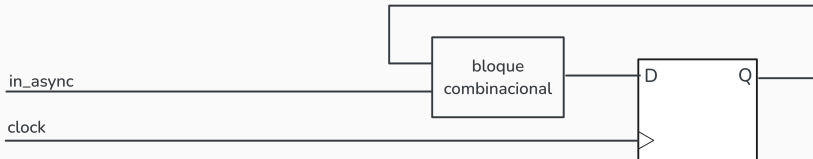
Violación de tiempo

Metaestabilidad. Entrada asincrónica



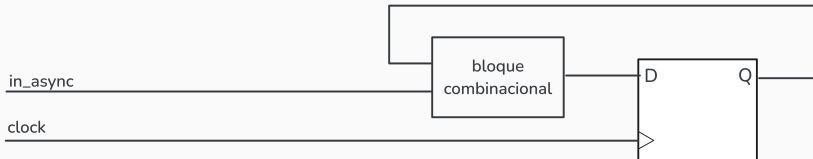
- No estamos dando margen de resolución ($T_r = 0$)

Metaestabilidad. Entrada asincrónica



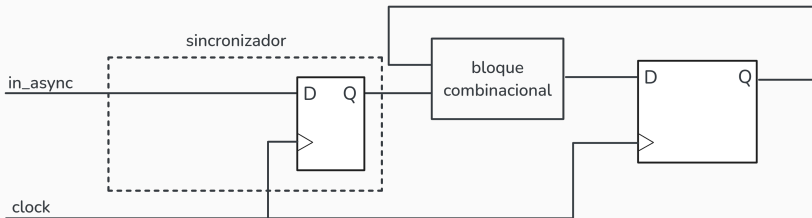
- No estamos dando margen de resolución ($T_r = 0$)
- MTBF (Mean Time Between Failures) es muy bajo

Metaestabilidad. Entrada asincrónica



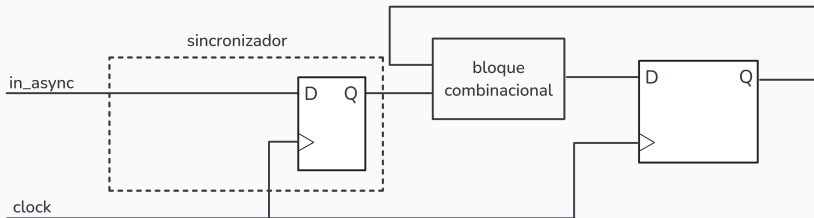
- No estamos dando margen de resolución ($T_r = 0$)
- MTBF (Mean Time Between Failures) es muy bajo
- Podemos agregar un segundo flip-flop para reducir la probabilidad de metaestabilidad

Metaestabilidad. Sincronizador simple



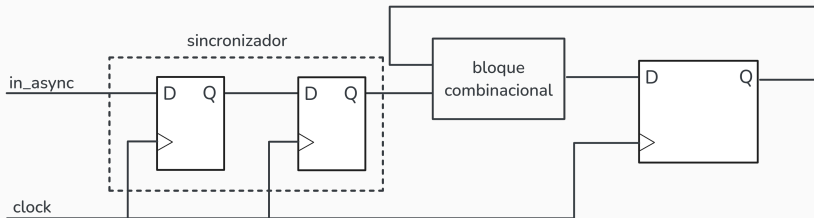
- $T_r = T_{clk} - (T_{comb} + T_{setup})$

Metaestabilidad. Sincronizador simple



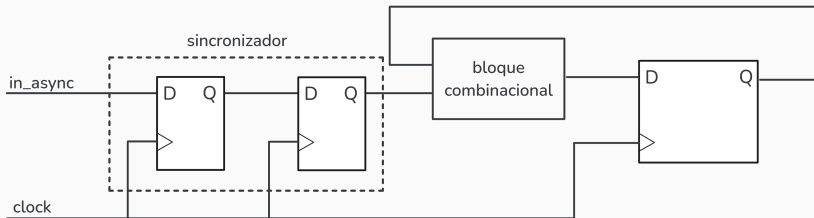
- $T_r = T_{clk} - (T_{comb} + T_{setup})$
- Aceptable si $T_{comb} \ll T_{clk}$

Metaestabilidad. Sincronizador doble



- $T_r = T_{clk} - T_{setup}$

Metaestabilidad. Sincronizador doble



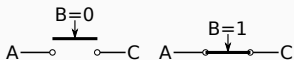
- $T_r = T_{clk} - T_{setup}$
- Esta solución satisfactoria en la mayoría de las aplicaciones

Registros y memorias

Pero antes... Componentes de Tres Estados

Buffer de Tres Estados

Noción Eléctrica



Símbolo

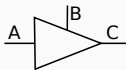
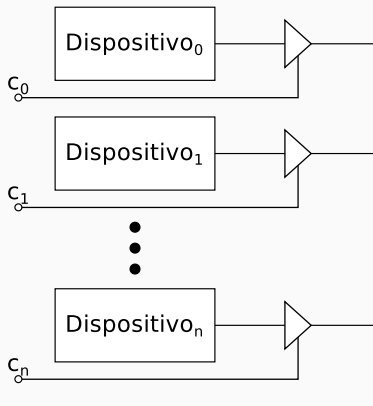


Tabla de Verdad

A	B	C
0	1	0
1	1	1
-	0	Hi-Z

Hi-Z significa “alta impedancia”, es decir, que tiene una resistencia alta al pasaje de corriente. Como consecuencia de esto, podemos considerar al pin C como desconectado del circuito.

Componentes de Tres Estados



IMPORTANTE: Sólo deben ser usados a la salida de componentes para permitirles conectarse a un medio compartido (bus).

Ejercicio 0

- a) Diseñar un registro de N *bits*. El mismo debe contar con N entradas $D_0, \dots, D_N$ para ingresar el dato a almacenar, N salidas $Q_0, \dots, Q_N$ para ver el dato almacenado y las señales de control CLK, RST y WREN para permitir la escritura del dato.

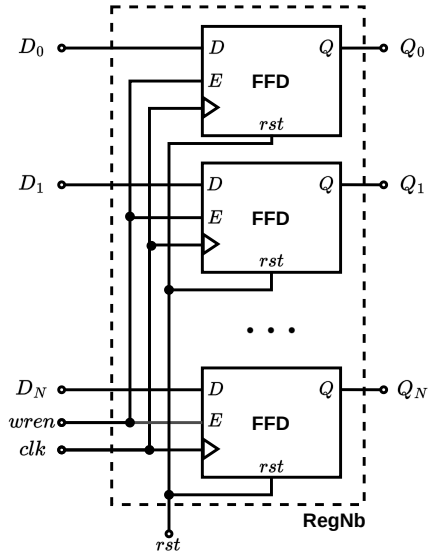
Ejercicio 0

- a) Diseñar un registro de N *bits*. El mismo debe contar con N entradas $D_0, \dots, D_N$ para ingresar el dato a almacenar, N salidas $Q_0, \dots, Q_N$ para ver el dato almacenado y las señales de control CLK, RST y WREN para permitir la escritura del dato.
- b) Modificar el diseño anterior agregándole componentes de 3 estados para que sólo cuando se active la señal de control EN (*enable* o habiliación) muestre el dato almacenado.

Ejercicio 0

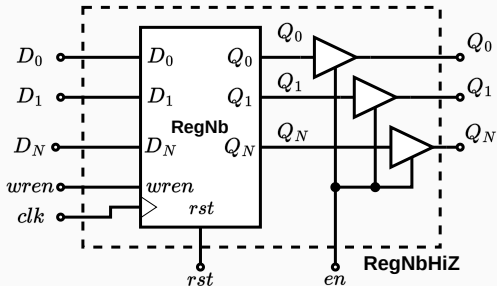
- a) Diseñar un registro de N *bits*. El mismo debe contar con N entradas $D_0, \dots, D_N$ para ingresar el dato a almacenar, N salidas $Q_0, \dots, Q_N$ para ver el dato almacenado y las señales de control CLK, RST y WREN para permitir la escritura del dato.
- b) Modificar el diseño anterior agregándole componentes de 3 estados para que sólo cuando se active la señal de control EN (*enable* o habiliación) muestre el dato almacenado.
- c) Modificar nuevamente el diseño para que D_i y Q_i estén conectadas entre sí al mismo tiempo teniendo en lugar de N entradas y N salidas, N entrada-salidas.

Solución - Ejercicio 0.a



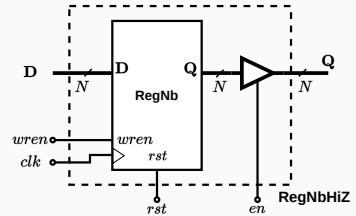
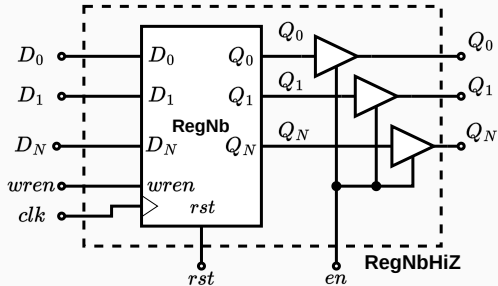
Solución - Ejercicio 0.b

Usamos el componente anterior:



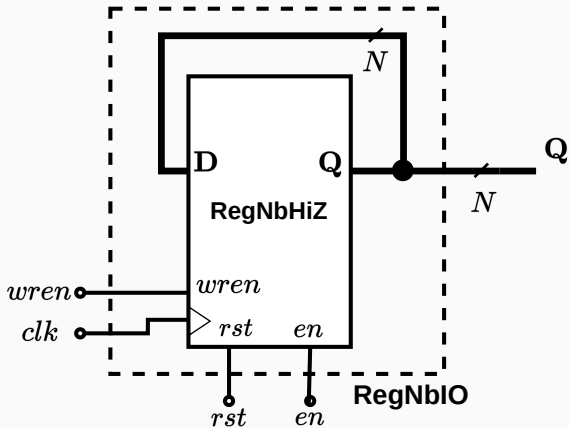
Solución - Ejercicio 0.b

Usamos el componente anterior:



Más prolijo :)

Solución - Ejercicio 0.c



Ejercicio 1

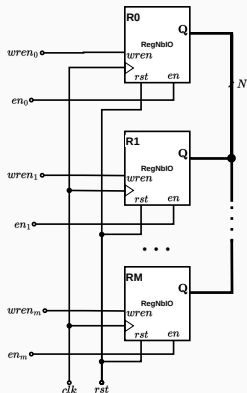
- a) Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.

Ejercicio 1

- a) Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
- b) Dar una secuencia de valores de las señales de control para que se copie el dato del $R1$ al $R0$.

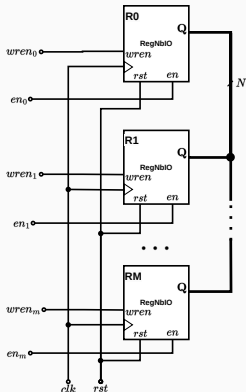
Ejercicio 1

- Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
- Dar una secuencia de valores de las señales de control para que se copie el dato del $R1$ al $R0$.



Ejercicio 1

- Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
- Dar una secuencia de valores de las señales de control para que se copie el dato del $R1$ al $R0$.

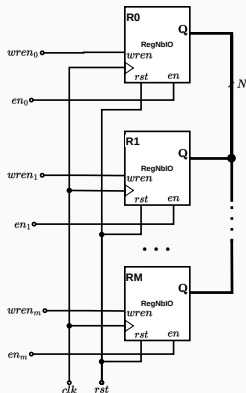


Señales de control:

R0	R1	...	RM
$wren_0$	$wren_1$	...	$wren_M$
rst	rst	...	rst
en_0	en_1	...	en_M

Ejercicio 1

- Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
- Dar una secuencia de valores de las señales de control para que se copie el dato del $R1$ al $R0$.



Señales de control:

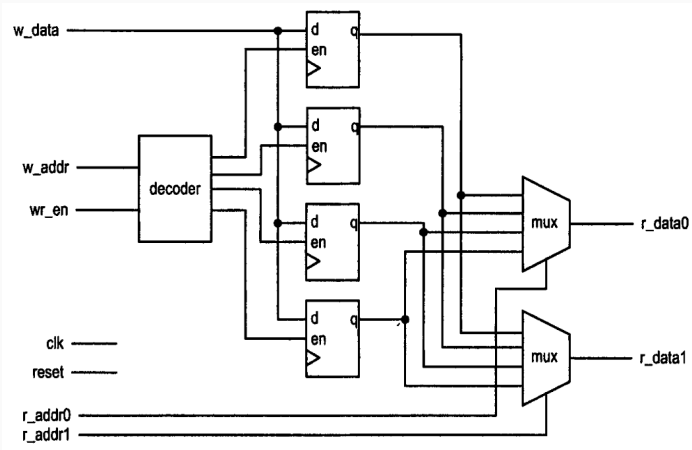
R0	R1	...	RM
$wren_0$	$wren_1$	...	$wren_M$
rst	rst	...	rst
en_0	en_1	...	en_M

Inician todas las señales en 0. Luego se sigue la siguiente secuencia:

- $en_1 \leftarrow 1$
- $wren_0 \leftarrow 1$
- ...clk....
- $wren_0 \leftarrow 0$
- $en_1 \leftarrow 0$

Register file

Ahora podemos tener varios registros de tamaño N y seleccionar de cuál leer o escribir.



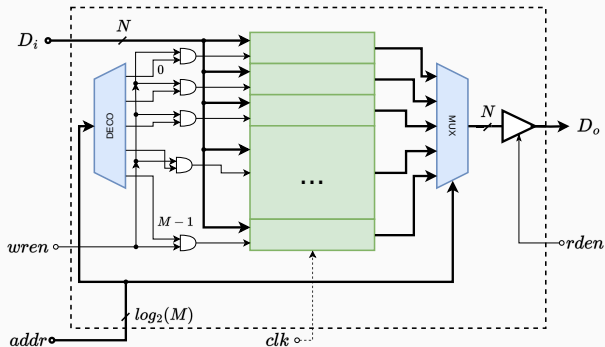
Register File

Memorias (intro)

Extendiendo la idea, **conceptualmente** podemos pensar una memoria como M posiciones de almacenamiento de N bits cada una.

Memorias (intro)

Extendiendo la idea, **conceptualmente** podemos pensar una memoria como M posiciones de almacenamiento de N bits cada una.



Conclusiones

- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales
- Analizamos el uso de un *clock* para limitar la cantidad de estados, controlar las transiciones y evitar carreras
- Vimos algunas implementaciones de flip-flops, registros y memorias

- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales
- Analizamos el uso de un *clock* para limitar la cantidad de estados, controlar las transiciones y evitar carreras
- Vimos algunas implementaciones de flip-flops, registros y memorias

Conclusiones

- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales
- Analizamos el uso de un *clock* para limitar la cantidad de estados, controlar las transiciones y evitar carreras
- Vimos algunas implementaciones de flip-flops, registros y memorias

- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales
- Analizamos el uso de un *clock* para limitar la cantidad de estados, controlar las transiciones y evitar carreras
- Vimos algunas implementaciones de flip-flops, registros y memorias

Básica

- *Digital Design and Computer Architecture*, David Harris, Sarah Harris
- *Digital Design: Principles and Practices*, John F. Wakerly

Avanzada

- *RTL Hardware Design Using VHDL*, Pong P. Chu